

SOCIETY ORDERING & POS SYSTEM

Complete Implementation Plan

Document Version:	1.0
Created:	April 18, 2026
Type:	Technical Implementation Plan
Scope:	Full-Stack Web Application
Target Users:	Customers, Staff, Admin

TABLE OF CONTENTS

1. Executive Summary
2. Project Overview & Objectives
3. System Architecture
4. Technology Stack
5. Database Design
6. Backend Development
7. Frontend Development
8. Admin Panel Development
9. API Endpoints Specification
10. Security & Authentication
11. Testing Strategy
12. Deployment & DevOps
13. Project Timeline & Milestones
14. Resource Requirements
15. Risk Assessment & Mitigation

1. EXECUTIVE SUMMARY

This document outlines the complete implementation plan for a private society-based ordering and POS (Point of Sale) system. The system enables residents to order café and grocery items online while providing staff with tools to manage both online and on-site orders. An admin panel controls user access and monitors system activity.

Key Objectives:

- Fast, frictionless ordering within a small community
- Unified system for online + offline (POS) orders
- Real-time order management by staff
- Role-based access control (Customer, Staff, Admin)
- Simple, maintainable codebase with clear logic
- Production-ready within 8-10 weeks

2. PROJECT OVERVIEW & OBJECTIVES

2.1 Problem Statement

Residents of gated societies require a convenient way to order café and grocery items without leaving home. Currently, there is no unified system for ordering, managing deliveries, or handling walk-in purchases.

2.2 Solution Overview

A web-based ordering and POS system that combines online ordering (customer-initiated) and on-site POS (staff-initiated for walk-ins). Staff handle all operations—accepting, delivering, and managing orders. Admin controls user approval and system monitoring.

2.3 User Roles

Role	Permissions	Key Activities
Customer	Browse, order, review	Browse products, add to cart, place orders, view order history, leave re
Staff	Manage orders, POS, products	Accept orders, mark delivered, create POS orders, manage products
Admin	User approval, monitoring	Approve/block users, view all orders, monitor system, manage products

3. SYSTEM ARCHITECTURE

3.1 High-Level Architecture

The system follows a three-tier architecture:

Frontend Layer (React/Next.js)

- Customer portal: Browse, cart, orders, reviews
- Staff dashboard: Order management, POS, product management
- Admin panel: User approval, system monitoring

Backend API Layer (Node.js + Express)

- RESTful API with role-based authorization
- Business logic: Order state machine, product availability
- Real-time notifications
- Payment integration hooks

Database Layer (PostgreSQL)

- Relational schema: Users, Products, Orders, OrderItems, Notifications, Reviews
- Transactional integrity for order processing
- Audit logs for admin monitoring

3.2 Core Entities & Relationships

Entity	Primary Purpose	Key Fields
USERS	Identity & access control	id, name, phone, role, status, building, flat
PRODUCTS	Inventory	id, name, price, is_available
ORDERS	Transaction hub	id, user_id, order_type, status, handled_by, total_price
ORDER_ITEMS	Order line items	order_id, product_id, quantity, price
NOTIFICATIONS	Event triggers	user_id, type, message, is_read
REVIEWS	Customer feedback	user_id, order_id, rating, comment

3.3 Order State Machine

Online Orders:

```

graph LR
    pending --> accepted
    accepted --> delivered
    accepted --> rejected

```

Onsite (POS) Orders:

pending → accepted (auto) → delivered (immediate)

Rules:

- First staff member to accept owns the order
- Only owner can deliver (no reassignment)
- Onsite orders skip manual acceptance and payment states

4. TECHNOLOGY STACK

Layer	Technology	Version	Purpose
Frontend	React / Next.js	18+	Server-side rendering, fast pages
Frontend	Tailwind CSS	3.x	Utility-first styling
Frontend	Axios / React Query	Latest	API communication, caching
Backend	Node.js	18+ LTS	Runtime
Backend	Express.js	4.x	HTTP framework
Backend	PostgreSQL	14+	Relational database
Backend	Sequelize / TypeORM	6.x	ORM
Auth	JWT / bcrypt	Latest	Session & password security
Real-time	Socket.io / Pusher	Latest	Order notifications
Deployment	Docker / Docker Compose	Latest	Containerization
Deployment	GitHub Actions	Latest	CI/CD pipeline
Hosting	AWS / Digital Ocean	Latest	Cloud infrastructure
Monitoring	LogRocket / Sentry	Latest	Error tracking & analytics

5. DATABASE DESIGN

5.1 Schema Overview

PostgreSQL with 6 core tables. All tables use UUID primary keys for security and scalability. Foreign key constraints enforce relational integrity.

5.2 Table Definitions

USERS

Column	Type	Constraints	Description
id	UUID	PK	Unique user identifier
name	VARCHAR(255)	NOT NULL	User full name
phone	VARCHAR(20)	UNIQUE	Phone number
role	ENUM	NOT NULL	customer / staff / admin
building	VARCHAR(100)		Building/block number
flat	VARCHAR(100)		Flat/unit number
status	ENUM	NOT NULL	pending / approved / blocked
created_at	TIMESTAMP	DEFAULT NOW()	Creation timestamp
updated_at	TIMESTAMP	DEFAULT NOW()	Last update

PRODUCTS

Column	Type	Constraints	Description
id	UUID	PK	Unique product identifier
name	VARCHAR(255)	NOT NULL	Product name
price	DECIMAL(10,2)	NOT NULL	Price in rupees
is_available	BOOLEAN	DEFAULT TRUE	Availability flag
created_at	TIMESTAMP	DEFAULT NOW()	Creation timestamp
updated_at	TIMESTAMP	DEFAULT NOW()	Last update

ORDERS

Column	Type	Constraints	Description
id	UUID	PK	Unique order identifier
user_id	UUID	FK NULLABLE	Customer (NULL for onsite)
order_type	ENUM	NOT NULL	online / onsite
status	ENUM	NOT NULL	pending / accepted / delivered / rejected
handled_by	UUID	FK	Staff member handling order
total_price	DECIMAL(10,2)	NOT NULL	Total order amount
payment_status	ENUM	DEFAULT unpaid	paid / unpaid
created_at	TIMESTAMP	DEFAULT NOW()	Order timestamp
updated_at	TIMESTAMP	DEFAULT NOW()	Last status change

5.3 Indexes & Query Optimization

- USERS: INDEX on (phone), INDEX on (status, created_at)
- PRODUCTS: INDEX on (is_available)
- ORDERS: INDEX on (user_id), INDEX on (status), INDEX on (handled_by), INDEX on (created_at)
- ORDER_ITEMS: INDEX on (order_id), INDEX on (product_id)

6. BACKEND DEVELOPMENT

6.1 API Structure

RESTful API following OpenAPI 3.0 specification. All endpoints require JWT authentication. Role-based authorization enforced at middleware level.

6.2 Core Modules

- **Auth Module**

- POST /auth/register - User registration

- POST /auth/login - User login (returns JWT)

- GET /auth/me - Current user profile

- POST /auth/logout - Invalidate token

- Token refresh mechanism (sliding expiration)

- **Products Module**

- GET /products - List all available products

- GET /products/:id - Get product details

- POST /products - Create product (Staff/Admin)

- PUT /products/:id - Update product

- DELETE /products/:id - Delete product (Admin)

- PATCH /products/:id/toggle - Toggle availability

- **Orders Module**

- GET /orders - User's orders

- POST /orders - Create online order

- GET /orders/:id - Order details

- PATCH /orders/:id/accept - Accept order (Staff)

- PATCH /orders/:id/deliver - Mark delivered (Staff)

- PATCH /orders/:id/reject - Reject order (Staff)

- POST /orders/onsite - Create POS order (Staff)

- **Notifications Module**

- GET /notifications - User's notifications

- PATCH /notifications/:id/read - Mark as read

- DELETE /notifications/:id - Delete notification

- WebSocket: /ws/notifications (real-time)

- **Admin Module**

- GET /admin/users/pending - Pending approvals

- PATCH /admin/users/:id/approve - Approve user

- PATCH /admin/users/:id/block - Block user

- GET /admin/orders - All orders

GET /admin/analytics - Revenue & stats

6.3 Business Logic Highlights

- Order Creation: Validate product availability, calculate totals, generate order record
- Order Acceptance: Lock order, assign to staff, prevent reassignment
- Delivery: Verify staff ownership, update status, trigger notifications
- Product Management: Soft-delete products, maintain audit trail
- Notification Dispatch: Async queue (Bull/RabbitMQ) for reliability
- Error Handling: Graceful degradation, detailed error codes, logging

6.4 Middleware & Security

- JWT Authentication: Verify token signature, check expiration
- Role-Based Authorization: Guard routes by role enum
- Input Validation: Sanitize & validate all inputs (Joi/Yup)
- Rate Limiting: Prevent abuse (express-rate-limit)
- CORS Configuration: Restrict to approved origins
- Request Logging: Log all API calls for audit trail

7. FRONTEND DEVELOPMENT

7.1 Architecture

Next.js 14+ with App Router. TypeScript for type safety. Component-based architecture with custom hooks for state management. Tailwind CSS for responsive design.

7.2 Customer Portal

- Home / Product Browse - Grid of available products with search/filter
- Product Detail - Description, pricing, add to cart, reviews
- Shopping Cart - Line items, quantity adjustment, checkout
- Checkout - Order review, delivery address, submit
- My Orders - Order list with status, order detail with timeline
- Leave Review - Star rating, text feedback
- Notifications - Real-time order updates
- Profile - Name, phone, address, preferences

7.3 Staff Dashboard

- Orders Dashboard - Pending/accepted/delivered tabs, real-time list
- Order Detail - Full details, accept/deliver buttons, customer info
- POS / New Order - Product selector, cart, payment, submit
- Product Management - Add/edit/toggle availability
- Stats - Daily orders, revenue, common items

7.4 Key Components

- **Navigation:** Role-aware navbar, logout, notification bell
- **ProductCard:** Reusable product display with add-to-cart
- **OrderTimeline:** Status progression visualization
- **CartSummary:** Item count, total, empty state
- **NotificationToast:** Real-time updates using Socket.io
- **Modal:** Confirmation, error, review forms
- **Loading States:** Skeleton screens, spinners, disabled states
- **Error Boundaries:** Graceful error handling

7.5 State Management

React Query (TanStack Query) for server state management and caching. Context API for global UI state (auth, theme). Custom hooks for business logic extraction.

7.6 Performance Optimizations

- Code splitting: Dynamic imports for modules

- Image optimization: Next.js Image component with lazy loading
- Caching: React Query cache management, HTTP caching headers
- Compression: Gzip, Brotli compression at CDN level
- Monitoring: Core Web Vitals tracking with Vercel Analytics

8. ADMIN PANEL DEVELOPMENT

8.1 Overview

Simplified admin interface for user approval and system monitoring. Access restricted to admin role. Built with same tech stack as customer/staff interfaces.

8.2 Key Screens

- **User Approval Queue:** List of pending users, approve/block/delete, audit log
- **User Management:** All users, view/edit roles, block/unblock, export
- **System Overview:** Key metrics, order count, revenue, active users
- **All Orders:** Read-only view of all orders, search/filter, export
- **Activity Log:** User actions, API calls, errors, system events
- **Product Management:** Add/edit/delete products, bulk upload, pricing
- **Revenue Report:** Daily/monthly/yearly breakdown, payment status
- **Settings:** Notification preferences, system configuration

8.3 Implementation Details

Data Tables: TanStack React Table for rich data manipulation

Charts: Recharts for revenue/order visualizations

Exports: CSV export for reports (PapaParse)

Real-time Updates: WebSocket for live user/order metrics

9. API ENDPOINTS SPECIFICATION

9.1 Base Structure

Base URL: **https://api.societyorder.local**

Version: **v1**

Authentication: **Bearer {JWT_TOKEN}**

Content-Type: **application/json**

9.2 Response Format

Success Response (200):

```
{
  "success": true,
  "data": {...},
  "message": "Operation successful"
}
```

Error Response (4xx/5xx):

```
{
  "success": false,
  "error": "ERROR_CODE",
  "message": "Human-readable message",
  "statusCode": 400
}
```

9.3 Key Endpoints (Full Specification)

POST /auth/register - Register new user

Params: name, phone, role (customer only), building, flat

Response: {user: {...}, token: 'jwt'}

POST /auth/login - Login user

Params: phone, password

Response: {user: {...}, token: 'jwt'}

POST /orders - Create online order

Params: items: [{product_id, qty}]

Response: {id, user_id, status: 'pending', total_price}

PATCH /orders/:id/accept - Accept pending order (Staff)

Params: None

Response: {id, status: 'accepted', handled_by}

PATCH /orders/:id/deliver - Mark order delivered (Staff only owner)

Params: None

Response: {id, status: 'delivered', updated_at}

POST /orders/onsite - Create POS order (Staff)

Params: items: [{product_id, qty}]

Response: {id, order_type: 'onsite', status: 'accepted', payment_status: 'paid'}

GET /products - List available products

Params: search, limit, offset

Response: [{id, name, price, is_available}, ...]

GET /admin/users/pending - Get pending user approvals (Admin)

Params: limit, offset

Response: [{id, name, phone, role, status}, ...]

PATCH /admin/users/:id/approve - Approve user (Admin)

Params: None

Response: {id, status: 'approved', updated_at}

GET /admin/orders - Get all orders (Admin)

Params: status, from_date, to_date, limit, offset

Response: [{id, user_id, status, total_price, created_at}, ...]

10. SECURITY & AUTHENTICATION

10.1 Authentication Flow

1. User registers with phone number
2. Backend creates user with status='pending'
3. Admin approves user → status='approved'
4. User logs in with phone + password
5. Backend validates, generates JWT (expires in 24h)
6. Frontend stores JWT in httpOnly cookie
7. All API calls include Authorization header

10.2 JWT Configuration

Algorithm: HS256 (HMAC SHA-256)
Secret: Strong 256-bit key from environment
Expiration: 24 hours
Refresh: Sliding window (extends on activity)
Claims: user_id, role, status, iat, exp

10.3 Password Security

Hashing: bcrypt with 12-round salt
Validation: Min 8 characters, alphanumeric
Reset: Email/SMS link with time-limited token

10.4 Authorization & Access Control

Role-Based Access Control (RBAC):

- Customer: Access own orders, reviews, products
- Staff: Accept/deliver orders, manage products, create POS orders
- Admin: User approval, system monitoring, all product management

Ownership Checks:

- Customers can only view their own orders
- Staff can only deliver orders they accepted
- Admin has read-only access to all data

10.5 Data Protection

HTTPS/TLS: All traffic encrypted in transit
CORS: Restrict to allowed origins
SQL Injection: Parameterized queries via ORM
XSS Prevention: Input sanitization, CSP headers
CSRF: Token validation for state-changing requests
Rate Limiting: 100 requests/minute per IP

10.6 Sensitive Data Handling

- Phone numbers: Indexed for search, never logged
- Passwords: Never stored in logs or backups
- Payment info: Never stored (PCI-DSS via gateway)
- Tokens: httpOnly, Secure, SameSite cookies

10.7 Audit & Compliance

All API calls logged with timestamp, user, action, IP
User approval/blocking logged for compliance